COLLEGE PROJECT DOCUMENTATION

RAG Assistant: A Retrieval-Augmented Generation Document Assistant

Project Estimation Document

Document	Project Estimation Document
Version	1.0
Date prepared	12 October 2026
Prepared by	
Course / Supervisor	

Planning document. The project is estimated from Day 0: no features are assumed to be built, and every activity in this document is planned work, not completed work.

Contents

1. Project Overview
2. Assumptions
3. Technology Stack
4. Work Breakdown Structure (WBS)
5. Effort Estimation
6. Cost Estimation
7. Resource Requirements
8. Risk Assessment
9. Estimation Summary

> All figures in this document are estimates made before development starts (Day 0). They are not measurements of completed work. The same task list and hours are used in the companion Project Plan document.

1. Project Overview

1.1 Project title

RAG Assistant: A Retrieval-Augmented Generation Document Assistant

1.2 Problem statement

Students and professionals often need answers that are buried in long PDFs, notes and reports. Reading every document is slow, keyword search misses answers that use different wording, and general-purpose chatbots can invent facts that are not in the user's documents. There is a need for an assistant that answers questions *only* from the user's own documents and shows exactly where each answer came from.

1.3 Project objectives

1. Let a signed-in user upload PDF, TXT and Markdown documents and manage them (list, delete).
2. Parse documents, split them into overlapping chunks and store vector embeddings in Qdrant.
3. Answer natural-language questions using retrieved chunks and an LLM, restricted to the user's documents.
4. Return citations (document, page, chunk) for every answer so the user can verify it.
5. Improve retrieval quality with hybrid search (dense + BM25 with Reciprocal Rank Fusion) and cross-encoder reranking.
6. Keep conversation history so users can ask follow-up questions and revisit past chats.
7. Protect each user's data with authentication and per-user isolation.
8. Deliver tested software with a README, UML diagrams and project documentation.

1.4 Intended users

- **Students** studying from lecture notes, papers and textbooks.
- **Researchers and professionals** who need quick, verifiable answers from reports and manuals.

- **Evaluators** (supervisor, examiners) who review the demonstration and documentation.

1.5 Scope and deliverables

In scope: user registration and login; document upload (PDF, TXT, MD), parsing, chunking, embedding and indexing; document list and delete; question answering with citations and sources; hybrid retrieval and reranking (planned enhancements); conversation history; a web frontend; automated tests; documentation.

Out of scope: OCR for scanned PDFs, other file formats (DOCX, PPTX, images), streaming responses, sharing documents between users, admin dashboards, fine-tuning models, mobile apps and production-scale deployment.

Deliverable	Description
Working application	FastAPI backend and Next.js frontend that run locally with Docker-hosted PostgreSQL and Qdrant
Source code	Git repository with backend, frontend and tests
README and setup guide	Installation, configuration (.env) and run instructions
UML diagrams	Class diagram and activity diagram
Project documents	This estimation document and the project plan
Test evidence	Automated test output and screenshots of key flows
Demonstration	Live or recorded walkthrough; public deployment only if required

2. Assumptions

1. One student works independently and performs all roles (developer, tester, documenter).
2. The project is estimated from Step 0 (Day 0). No existing code or completed features are counted as done.
3. Suitable open-source libraries and managed services may be used instead of building components from scratch (for example pypdf, sentence-transformers, Qdrant, an LLM API).
4. The system is for a college assignment and demonstration, not production-scale use: a few users, tens of documents, files up to about 50 MB.
5. The student already knows basic Python, JavaScript/TypeScript, SQL and Git. Time to learn these from scratch is not included.
6. Planned working time is 3 focused hours per working day, 5 working days per week.
7. A laptop with at least 8 GB RAM (16 GB recommended), Docker and internet access is available.
8. An LLM API account with a small amount of credit is available; embeddings and reranking run locally for free.
9. Documents contain selectable text. Scanned image-only PDFs are not supported (no OCR).
10. Deployment to a public host is optional and its effort (about 4-6 hours) is **not** included in the total.
11. Prices in the cost section are illustrative and could not be verified at the time of writing.

3. Technology Stack

Area	Technology	Why it is used
Frontend	Next.js, React, TypeScript, Tailwind CSS	Component-based UI with routing; TypeScript catches API contract errors early; Tailwind speeds up consistent styling.
Backend API	Python, FastAPI, Uvicorn, Pydantic	Python has the strongest ML/RAG library ecosystem; FastAPI gives typed request validation and automatic API docs.
Relational database	PostgreSQL (psycopg driver)	Reliable storage for users, document metadata, conversations and messages, with foreign keys and cascading deletes.
Vector database	Qdrant (qdrant-client)	Purpose-built similarity search with payload filters, so search can be restricted to one user or one document.
Document parsing	pypdf; Python standard library for TXT/MD	Extracts text page by page so citations can point to page numbers.
Embeddings	sentence-transformers (e.g. BAAI/bge-small-en-v1.5)	Free, runs locally on CPU, small enough for a laptop; no per-request cost.
Retrieval	Dense search in Qdrant; BM25 (rank-bm25); Reciprocal Rank Fusion; cross-encoder reranker	Dense search finds meaning, BM25 finds exact terms, RRF merges both rankings, the reranker reorders the best candidates.
LLM	OpenAI API (a small, low-cost model)	Generates the final answer from retrieved context; managed service avoids hosting a large model.
Authentication	PBKDF2 password hashing (hashlib), JWT (PyJWT)	Stateless token authentication that the Next.js client can send with each request.

Area	Technology	Why it is used
Testing	pytest, FastAPI TestClient/httpx; Node test runner, ESLint, tsc	Automated unit and API tests on the backend; type checking and lint on the frontend.
Dev tooling	Git, GitHub, Docker Compose, VS Code	Version control and reproducible local PostgreSQL and Qdrant containers.
Deployment (optional)	Vercel (frontend); a small cloud VM or container host (backend); managed PostgreSQL/Qdrant	Only if the assignment requires a public demo; local demonstration is the default.

4. Work Breakdown Structure (WBS)

The work is divided into 10 phases and 44 tasks. Tasks marked Enhancement are in the proposed scope but can be deprioritised if time runs short; all other tasks are core. Dependencies list the tasks that must be finished first.

ID	Task	Priority	Hours	Depends on
P1	**Requirements and environment setup**		**6**	
1.1	Requirements analysis, scope and feature list	Core	1.5	-
1.2	Project estimation and project plan (this documentation)	Core	1.5	1.1
1.3	Environment setup: Python, Node.js, Docker (PostgreSQL, Qdrant), API key	Core	1.5	1.1
1.4	Project skeleton: FastAPI app, settings/.env, logging, health endpoint, Git repo	Core	1.5	1.3
P2	**Architecture and database design**		**6**	
2.1	System architecture, REST API design and data flow	Core	2	1.1
2.2	PostgreSQL schema (users, documents, conversations, messages) and initialisation	Core	2.5	1.4, 2.1
2.3	Qdrant collection and payload design (user and document filters)	Core	1.5	1.4, 2.1
P3	**Document ingestion and parsing**		**8**	
3.1	Upload endpoint: file type, size and filename validation; file storage	Core	3	2.2
3.2	PDF parser (page-wise text extraction with pypdf)	Core	2	1.4
3.3	TXT and Markdown parser (encoding handling)	Core	1	1.4
3.4	Document metadata store; list, get and delete endpoints	Core	2	2.2, 3.1
P4	**Chunking, embeddings and vector indexing**		**10**	
4.1	Text chunking with overlap and page tracking	Core	2	3.2, 3.3
4.2	Embedding generation (sentence-transformers model)	Core	2	1.3
4.3	Qdrant store: upsert, filtered similarity search, delete	Core	3	2.3, 4.2
4.4	Ingestion pipeline orchestration and cleanup on failure	Core	2	3.1, 4.1, 4.3
4.5	Verify indexing with sample documents	Core	1	4.4
P5	**Retrieval and answer generation**		**12**	
5.1	Query endpoint with dense vector retrieval	Core	2.5	4.4
5.2	Prompt design and LLM integration (answers grounded in context)	Core	3.5	5.1
5.3	Hybrid retrieval: BM25 keyword search fused with Reciprocal Rank Fusion Enhancement	Optional	3	5.1
5.4	Cross-encoder reranking of retrieved chunks Enhancement	Optional	2	5.3
5.5	Retrieval check on sample questions; tune chunk size and top-k	Core	1	5.2
P6	**Citation generation**		**3**	
6.1	Context builder that labels chunks with source IDs [S1]..[Sn]	Core	1.5	5.2
6.2	Citation and source objects in the API response	Core	1.5	6.1

ID	Task	Priority	Hours	Depends on
P7	**Authentication and conversation history**		**9**	
7.1	User registration and login with password hashing	Core	3	2.2
7.2	JWT authentication, route protection, per-user data isolation	Core	2.5	7.1
7.3	Conversation and message storage; history endpoints	Core	2.5	2.2, 7.2
7.4	Use recent history in queries; rewrite follow-up questions Enhancement	Optional	1	5.2, 7.3
P8	**Frontend integration**		**21**	
8.1	Next.js, TypeScript and Tailwind CSS setup; application layout	Core	2	1.3
8.2	API client, login and registration pages, session handling	Core	4	7.2, 8.1
8.3	Document library: upload with progress, list, delete	Core	3.5	3.4, 8.2
8.4	Chat interface: messages, composer, loading/error states, document scope	Core	5	6.2, 8.2
8.5	Citation markers and source panel	Core	2	8.4
8.6	Conversation history sidebar	Core	1.5	7.3, 8.4
8.7	Frontend-backend integration fixes and UI polish	Core	3	8.3, 8.5, 8.6
P9	**Testing and error handling**		**15**	
9.1	Unit tests: parsers, chunking, retrieval fusion, reranking, citations	Core	4	P3-P6
9.2	API tests: auth, upload, documents, query, history, user isolation	Core	4	P7
9.3	End-to-end manual testing with sample documents	Core	3	8.7
9.4	Error-handling hardening and bug fixing	Core	4	9.1, 9.2, 9.3
P10	**Documentation, diagrams and final demonstration**		**9**	
10.1	README and setup instructions	Core	1.5	8.7
10.2	UML class diagram and activity diagram	Core	1.5	9.4
10.3	Test evidence and screenshots	Core	1.5	9.4
10.4	Final report; update estimation and plan with actual progress	Core	2	10.1-10.3
10.5	Demonstration preparation and rehearsal	Core	1.5	10.4
10.6	Final review and submission	Core	1	10.5
C	**Contingency (about 15% of planned work)**		**15**	-
	Total estimated effort		**114**	

5. Effort Estimation

Estimates are bottom-up: each task in the WBS was sized in hours for one student, then summed per phase. Testing, debugging, integration and report preparation are separate tasks rather than hidden inside other estimates, and a contingency of 15 hours (about 15% of the planned work) covers unexpected problems.

5.1 Effort by phase

Phase	Name	Tasks	Hours	Share
P1	Requirements and environment setup	4	6	5.3%
P2	Architecture and database design	3	6	5.3%
P3	Document ingestion and parsing	4	8	7.0%
P4	Chunking, embeddings and vector indexing	5	10	8.8%
P5	Retrieval and answer generation	5	12	10.5%
P6	Citation generation	2	3	2.6%
P7	Authentication and conversation history	4	9	7.9%
P8	Frontend integration	7	21	18.4%
P9	Testing and error handling	4	15	13.2%
P10	Documentation, diagrams and final demonstration	6	9	7.9%
	Subtotal: planned work	**44**	**99**	**86.8%**
C	Contingency (about 15% of planned work)	-	15	13.2%
	Total estimated effort		**114**	**100%**

5.2 Effort by activity type

Activity type	Hours	Share	Included tasks
Design and implementation	70.5	61.8%	All P1-P8 tasks except 1.2 and 8.7
Testing	11	9.6%	Tasks 9.1, 9.2, 9.3 (plus checks inside 4.5 and 5.5)
Debugging and integration	7	6.1%	Tasks 8.7 and 9.4
Documentation and report preparation	10.5	9.2%	Tasks 1.2 and 10.1-10.6
Contingency	15	13.2%	Unplanned debugging, rework, illness
Total	**114**	**100%**	

5.3 Core work versus enhancements

Category	Hours	Tasks
Core (mandatory) work	93	All tasks not marked Enhancement
Planned enhancements	6	5.3 BM25 + RRF hybrid retrieval; 5.4 cross-encoder reranking; 7.4 follow-up question rewriting
Contingency	15	Buffer
Total	114	

5.4 Estimation method and confidence

- Method: bottom-up task estimation with expert judgement, assuming mature libraries are used.
- Expected range: about 95-140 hours. Integration, PDF edge cases and frontend polish are the least certain parts.
- The estimate excludes learning a language or framework from scratch and optional deployment (about 4-6 h).

6. Cost Estimation

Development effort (student hours) and **monetary expenses** are different. The 114 hours of effort are unpaid student time and are not converted to money. Monetary costs come only from external services. The design keeps them low by running the databases and the embedding and reranker models locally.

Item	Basis	Estimated cost	Notes and assumptions
Development labour	114 person-hours of student time	Not a cash expense	Effort, not money. Shown for completeness; no notional hourly rate is applied.
Embedding and reranker models	Open-source models run locally	US$0	Downloaded once from Hugging Face; uses local CPU.
PostgreSQL and Qdrant	Local Docker containers	US$0	Free open-source images on the developer laptop.
LLM API (OpenAI)	Pay-per-token, usage-dependent	About US$1-10 for the whole project (illustrative)	Assumes a small model and a few hundred test and demo questions. Cost = input tokens x input price + output tokens x output price. Check current prices before use and set a spending limit.
Frontend hosting (optional)	Vercel Hobby or similar free tier	US$0 (free tier assumed)	Free tiers have usage limits and non-commercial terms; verify before deploying.
Backend hosting (optional)	Small VM or container host	About US$5-25 per month (illustrative)	Embedding and reranker models need roughly 1-2 GB RAM, which free tiers often do not provide.
Managed databases (optional)	Managed PostgreSQL and Qdrant Cloud free tiers	US$0 within free tiers (illustrative)	Small free clusters are typically enough for a demo; paid tiers are usage-dependent.
Domain name (optional)	Custom domain	About US$10-15 per year (illustrative)	Not needed; the default hosting subdomain is sufficient.

6.1 Cost summary

Scenario	Estimated monetary cost
Local development and local demonstration (default)	About US$1-10 in total, all of it LLM API usage (illustrative)
With optional public deployment	Above, plus about US$5-25 per month while the backend is hosted; US$0 for free-tier frontend and databases (illustrative)

> All prices are illustrative estimates, not confirmed prices. Provider prices and free-tier limits change, so check them on each provider's pricing page before use. LLM cost depends on usage: number of questions x (prompt tokens + answer tokens) x the model's price per token.

7. Resource Requirements

Category	Requirement
Human resources	1 student (developer, tester and documenter); periodic guidance from the supervisor
Hardware	Laptop or desktop: 8 GB RAM minimum (16 GB recommended), about 10 GB free disk for Docker images and models, multi-core CPU (GPU not required); internet connection
Software	Python 3.12, Node.js (LTS), Docker Desktop with Docker Compose, Git, VS Code or similar editor, a modern web browser
Libraries	FastAPI, Pydantic, psycopg, qdrant-client, pypdf, sentence-transformers, rank-bm25, openai, PyJWT, pytest; Next.js, React, TypeScript, Tailwind CSS
Accounts	LLM API account (e.g. OpenAI) with billing limit; GitHub account; optional: Vercel and cloud hosting accounts
External services	LLM API (required); Hugging Face model downloads (one-time); optional managed PostgreSQL / Qdrant Cloud / hosting
Test data	A small set of sample PDF, TXT and MD documents with known content, and a list of sample questions with expected sources

8. Risk Assessment

Likelihood and impact are rated Low, Medium or High.

ID	Risk	Likelihood	Impact	Mitigation
R1	Effort underestimated; schedule slips	Medium	High	15 h contingency; weekly milestone review; drop enhancements first (deprioritisation order).
R2	LLM API problems: missing key, cost, rate limits, outage	Medium	Medium	Use a small model and a spending cap; return a clear error message; mock the LLM in automated tests.
R3	Poor text extraction (scanned PDFs, tables, multi-column layouts)	High	Medium	Declare OCR out of scope; test several PDF types early; handle documents that yield no text.
R4	Slow or heavy local models (download size, CPU-only inference)	Medium	Medium	Use small models; load them once and cache; run ingestion off the request event loop; limit file size.
R5	Weak retrieval or answers that are not grounded	Medium	High	Hybrid retrieval and reranking; strict context-only prompt; check citations on a fixed set of sample questions.
R6	Environment issues (Docker, Windows paths, ports)	Medium	Medium	Set up Docker Compose on Day 1; document setup steps in the README as they are solved.
R7	Security flaw: cross-user data access or weak authentication	Low	High	Filter every query by user ID; hash passwords; expire tokens; write user-isolation API tests.
R8	Frontend-backend contract mismatches	Medium	Medium	Define the API in Phase 2; mirror response models in TypeScript types; integration buffer in task 8.7.
R9	Inconsistent data between PostgreSQL and Qdrant after a failure	Low	Medium	Remove the file and vectors when an upload fails; delete from both stores when a document is deleted.
R10	Scope creep (extra file types, features, styling)	Medium	Medium	Fixed scope and exclusion list; new ideas go to a 'future work' list.
R11	Single developer unavailable (illness, exams)	Medium	High	Contingency days at the end; finish core features first so a working demo exists early.

9. Estimation Summary

114 h
Total effort incl. 15 h contingency

93 h
Core (mandatory) planned work

6 h
Optional enhancements

38 days
Working days at 3 h/day (about 8 weeks)

Item	Estimate
Total person-hours	114 hours (99 h planned work + 15 h contingency)
Core versus optional	93 h core work; 6 h optional enhancements
Calendar duration	38 working days at 3 h/day, about 8 calendar weeks (5 working days per week)
Expected deliverables	Working application, source code, README, UML class and activity diagrams, estimation and plan documents, test evidence, demonstration
Monetary cost	About US$1-10 for LLM API usage (illustrative); optional hosting extra
Main assumptions	One student; Day 0 start; existing libraries used; local demo; text-based documents; 3 h/day
Main limitations	Estimates are not measurements; prices unverified; no OCR; optional deployment not included

> **Schedule baseline check.** The suggested baseline of 14 working days at 2-3 hours per day provides only 28-42 hours. The bottom-up estimate is 114 hours, so the baseline cannot hold the full scope. The plan therefore uses 38 working days at 3 hours per day (about 8 weeks of 5 working days). Finishing in 14 working days would need about 9 hours per day, or a reduced scope (see the deprioritisation order).